Cloud Based Data Engineering

Lecture 10: Document Clustering and Topic Modeling

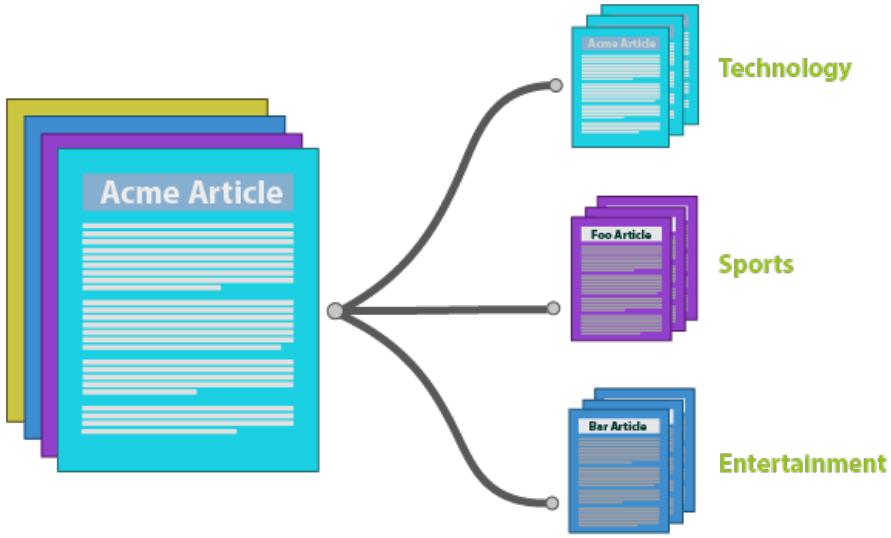
Introduction

- A corpus is the group of all the text documents whereas a document is a collection of paragraphs.
- A paragraph is a collection of sentences, and a sentence is a sequence of words (or tokens) in a grammatical construct.
- **Topics or themes** are a group of **statistically significant** “tokens” or words in a “corpus”.

Corpus **Document** **Token**

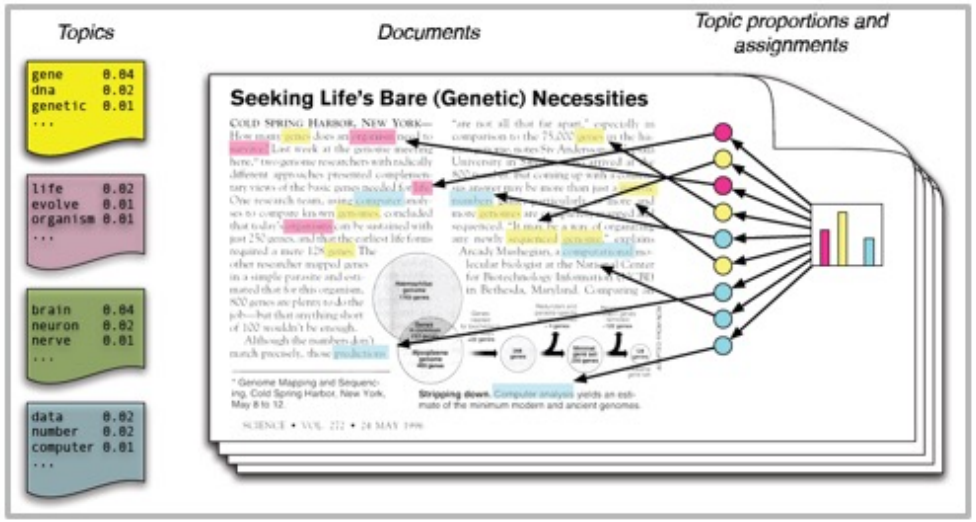
Document Clustering

*The task of organizing a collection of **documents**, whose classification is unknown, into meaningful groups (clusters) that are homogeneous according to some notion of proximity (distance or similarity) among **documents**.*



Topic Modeling

A machine learning technique that automatically analyzes text data to determine cluster words for a set of documents. This is known as 'unsupervised' machine learning because it doesn't require a predefined list of tags or training data that's been previously classified.



PBL – Project Based Learning

Project-based learning (PBL) or project-based instruction is an instructional approach designed to give students the opportunity to develop knowledge and skills through engaging projects set around challenges and problems they may face in the real world.

Discriminative vs. Generative Models

Discriminative Models - type of logistical model and are mostly used for supervised learning.

Its use conditional probabilities to predict.

For example,

Logistic Regression

Decision Tree

Random Forest

Support Vector Machine (SVM)

Traditional Neural Network

Discriminative vs. Generative Models

Generative Models - use statistics to generate or create new data.

- Estimate the probabilities using the joint probability distribution P(X, Y).
- Estimate the data points and differentiates the classes based on these computed probabilities of the class labels.

For example,

Hidden Markov Model (HMM)

Latent Dirichlet Allocation (LDA)

Generative Adversarial Networks (GANs)

Autoencoders

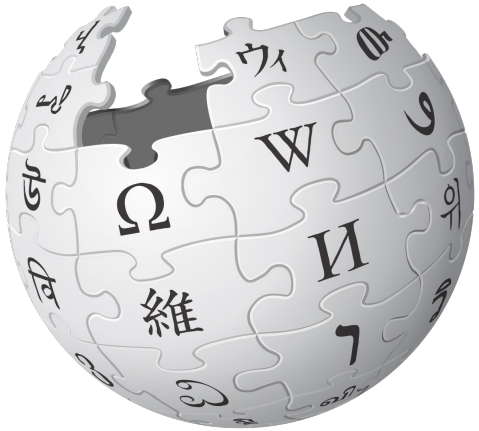
Document Clustering and Topic Modeling

Unsupervised learning models to cluster unlabeled documents into different groups

1. Load Data
2. Tokenizing and Stemming
3. TF-IDF
4. Document Similarity
5. K-Means Clustering for Documents: naïve vs. hierarchical
6. Latent Dirichlet Allocation (LDA) for Topic Modeling

The Dataset

- title_list.txt: List of all movie's titles
- Synopses_list_wiki.txt: synopsis of each movie (in the same order) from Wikipedia
- Synopses_list_imdb.txt: synopsis of each movie (in the same order) from IMDB



Step 1: Load and Merge

1. Load each data source to a variable.
2. Merge the synopses from Wiki and IMDB of the same movie.

Note: keep the sources in text format for step 2. In case we want to use a library (Pandas, Spark, etc.), we will do it after processing.

Step 2: Tokenizing and Stemming

Tokenization

Input: "*Friends, Romans and Countrymen*"

Output: (Tokens) *Friends, Romans, Countrymen*

Issues in tokenization:

Finland's capital $\rightarrow$ ***Finland*** AND ***s***? ***Finlands***? ***Finland's***?

San Francisco: one token or two? How do you decide it is one token?

Step 2: Tokenizing and Stemming – Stop Words

With a stop list, you exclude from the dictionary entirely the commonest words.

- They have little semantic content: *the, a, and, to, be*

- There are a lot of them: ~30% of postings for top 30 words

Lemmatization - Reduce variant forms to base form

am, are, is $\rightarrow$ *be*

car, cars, car's, cars' $\rightarrow$ *car*

the boy's cars are different colors $\rightarrow$ *the boy car be different color*

Step 2: Tokenizing and Stemming

Stemming - Reduce terms to their “roots” before indexing

automate(s), automatic, automation all reduced to ***automat***.

for example compressed and compression are both accepted as equivalent to compress.

for exampl compress and compress ar both accept as equival to compress

Step 2: Tokenizing and Stemming

- Do we need to implement it from scratch?
- How many stop words are exists?
- Tokenizing is easy (split), but stemming become more challenging

The solution: NLTK – Natural Language ToolKit for Python

Step 3: TF-IDF

TF-IDF (term frequency-inverse document frequency) is a statistical measure that evaluates how relevant a word is to a document in a collection of documents.

Dome by multiplying two metrics:

1. how many times a word appears in a document
2. The inverse document frequency of the word across a set of documents.

Terminology

TF-IDF = Term Frequency (TF) * Inverse Document Frequency (IDF)

$$TF(t,d) = \frac{count\ of\ t\ in\ d}{number\ of\ words\ in\ d}$$

$$DF(t) = occurrence\ of\ t\ in\ N\ documents$$

$$IDF(t) = \log\left(\frac{N}{DF(t)}\right)$$

$$TF - IDF(t,d) = TF(t,d) \cdot \log\left(\frac{N}{DF(t)}\right)$$

Example

	going	to	today	i	am	it	is	rain
Doc 1	0	$0.58 \cdot 0.16 = 0.09$	0.09	0	0	0.25	0.25	0.25
Doc 2	0	0	0.09	0.09	0.09	0	0	0
Doc 3	0	$0.58 \cdot 0.12 = 0.06$	0	0.06	0.06	0	0	0

- “it”, “is”, “rain” important for doc1 but not for docs 2 and 3.
- Docs 1 and 2 talks about today

Step 3: TF-IDF

ignore terms that have a document frequency strictly higher than the given threshold

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_model = TfidfVectorizer(max_df=0.8, max_features=200000,
        min_df=0.2, stop_words='english',
        use_idf=True, tokenizer=tokenization_and_stemming,
        ngram_range=(1,1))
```

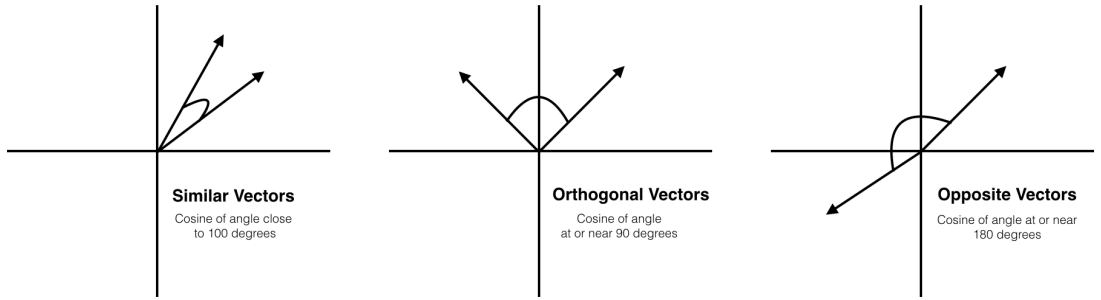
A function

N-gram (dependency in the previous word)

Step 4: Document Similarity

Cosine similarity - a measure of similarity between two sequences of numbers

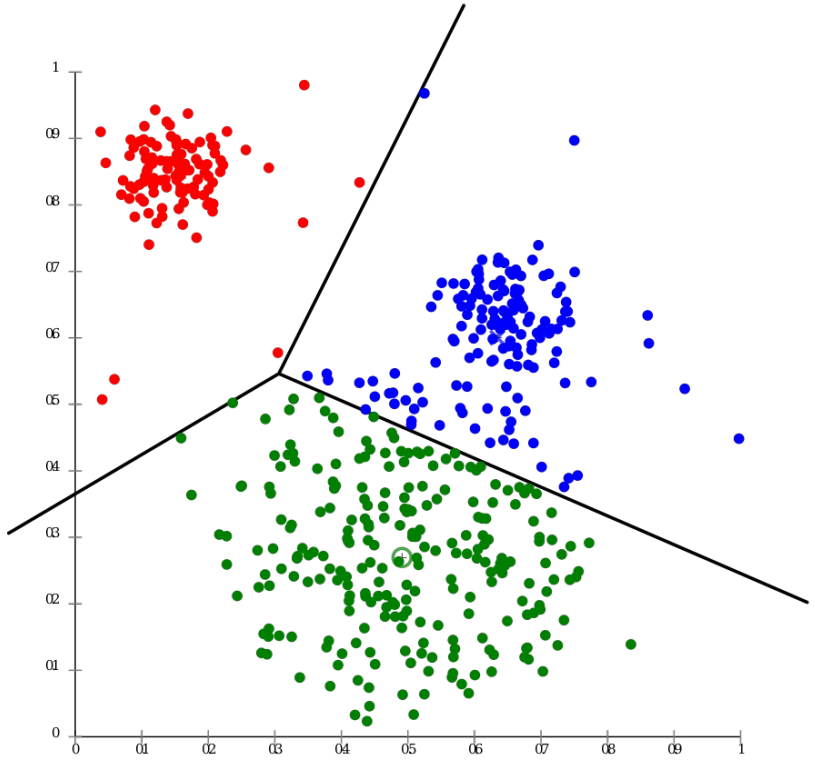
In our case, we represent each document as tf-idf values



$$Cosine(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Step 5: K-Means Clustering

Clustering is the task of grouping together a set of objects in a way that objects in the same cluster are more like each other than to objects in other clusters.



Step 5: K-Means Clustering

Clustering is the task of grouping together a set of objects in a way that objects in the same cluster are more like each other than to objects in other clusters.

Euclidean Distance

$$D(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_k - y_k)^2}$$

Manhattan Distance

$$D(x, y) = |x_1 - y_1| + |x_2 - y_2| + \cdots + |x_k - y_k|$$

Step 5: K-Means Clustering

How it works?

1. Choose k random points (from the dataset) as initial centers: $C_1, \dots, C_k$
2. Repeat until convergence:
    1. For each data point (x_i), calculate the distance from each $C_1, \dots, C_k$
    2. $x_i \in C_j$ if and only $D(x_i, C_j)$ is minimal
    3. Update centroids:

       For each C_j , the new center is $(\frac{1}{n}\sum x_1 , \frac{1}{n}\sum x_2 , \dots , \frac{1}{n}\sum x_n)$

Step 5: K-Means Clustering

Since clustering is an unsupervised problem, we will evaluate it by:

1. **Silhouette** – does the partition was good?
    1. Silhouette ≈ 1 – great partition
    2. Silhouette ≈ -1 – no patterns were found, and the partition is not accurate
    3. Silhouette ≈ 0 – the separation is controversial
2. **SSE** – Sum of Squared Error

    Sum of squared distances of samples to their closest cluster center

Step 5: K-Means Clustering

Finding the optimal k is an **NP-hard** problem, we need to use approximation.

We will use the **amplification** mechanism:

For $k = 1$ to m

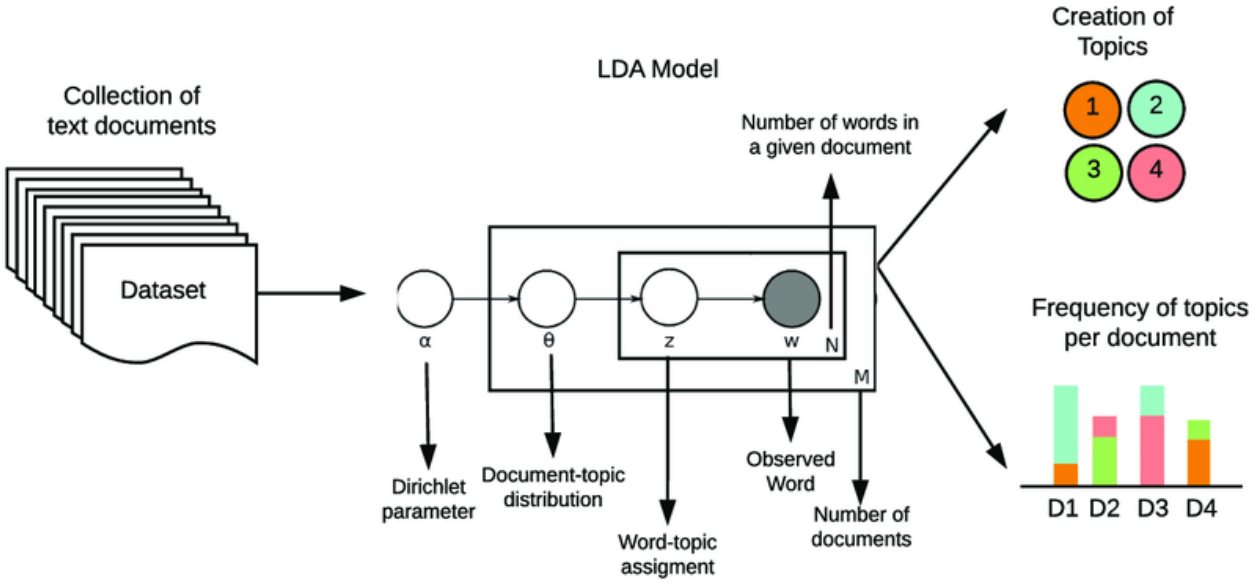
Train K-means with k clusters

Store the silhouette and SSE

Compare and choose the k where silhouette is maximal, and SSE is minimal.

Step 6: LDA for Topic Modeling

Latent Dirichlet Allocation (LDA) is a **stochastic process** of topic model and is used to classify text in a document to a particular topic (using a reverse engineering). It builds a topic per document model and words per topic model.



Step 6: LDA for Topic Modeling

Simple example: consider we have an urn with α **black** balls.

1. Each time we need an observation, we draw a ball from the urn.
2. If the ball is **black**, we generate a new (non-black) color uniformly, label a new ball this color, drop the new ball into the urn along with the ball we drew, and return the color we generated.
3. Otherwise, label a new ball with the color of the ball we drew, drop the new ball into the urn along with the ball we drew, and return the color we observed.

Step 6: LDA for Topic Modeling

The LDA makes two key assumptions:

1. Documents are a mixture of topics

2. Topics are a mixture of tokens (or words)

These topics (using the probability distribution) generate the words.

In statistical language, the **documents** are known as the **probability density** (or distribution) of **topics** and the **topics** are the **probability density** (or distribution) of **words**.

Step 6: LDA for Topic Modeling

Consider the following example:

Document 1: I want to watch a movie this weekend.
Document 2: I went shopping yesterday. New Zealand won the World Test Championship by beating India by eight wickets at Southampton.
Document 3: I don't watch cricket. Netflix and Amazon Prime have very good movies to watch.
Document 4: Movies are a nice way to chill however, this time I would like to paint and read some good books. It's been so long!
Document 5: This blueberry milkshake is so good! Try reading Dr. Joe Dispenza's books. His work is such a game-changer! His books helped to learn so much about how our thoughts impact our biology and how we can all rewire our brains.

Step 6: LDA for Topic Modeling

After preprocessing the documents (clean, preprocess and tokenize the text to words):

1. For $1 \leq i \leq 5$ the D_i is the i^{th} document

2. W – the total amount of words.

3. For our example, we assume $W = 8$

	W1	W2	W3	W4	W5	W6	W7	W8
D1	0	1	1	0	1	1	0	1
D2	1	1	1	1	0	1	1	0
D3	1	0	0	0	1	0	0	1
D4	1	1	0	1	0	0	1	0
D5	0	1	0	1	0	0	1	0

Document 1 include word 2

The matrix called **Document Word Matrix** (DWM).

Step 6: LDA for Topic Modeling

LDA converts DWM into two other matrices:

Document Topic Matrix and Topic Word Matrix

	W1	W2	W3	W4	W5	W6	W7	W8
D1	0	1	1	0	1	1	0	1
D2	1	1	1	1	0	1	1	0
D3	1	0	0	0	1	0	0	1
D4	1	1	0	1	0	0	1	0
D5	0	1	0	1	0	0	1	0

Shape: 5 * 8

Document Topic Matrix

	K1	K2	K3	K4	K5	K6
D1	1	0	0	0	0	0
D2	0	1	0	0	1	1
D3	1	1	0	0	0	0
D4	1	0	0	1	0	1
D5	0	0	1	1	0	0

Shape: 5 * 6

Topic Word Matrix

	W1	W2	W3	W4	W5	W6	W7	W8
K1	0	1	1	0	1	0	1	0
K2	1	1	1	1	0	1	1	1
K3	1	0	0	0	0	1	0	0
K4	1	1	0	1	1	0	0	1
K5	0	0	1	1	0	1	1	1
K6	1	0	1	1	1	0	0	1

Shape: 6 * 8

Step 6: LDA for Topic Modeling

Notations:

α – per-document topic distributions

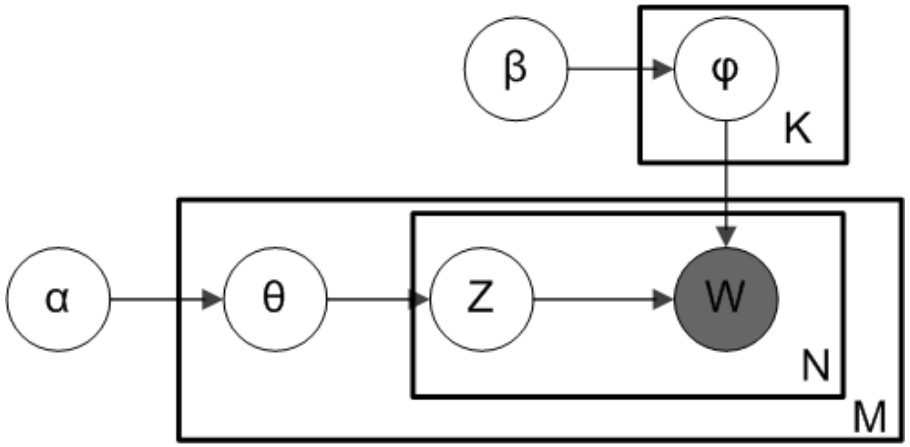
β – per-topic word distribution

θ – topic distribution for document m

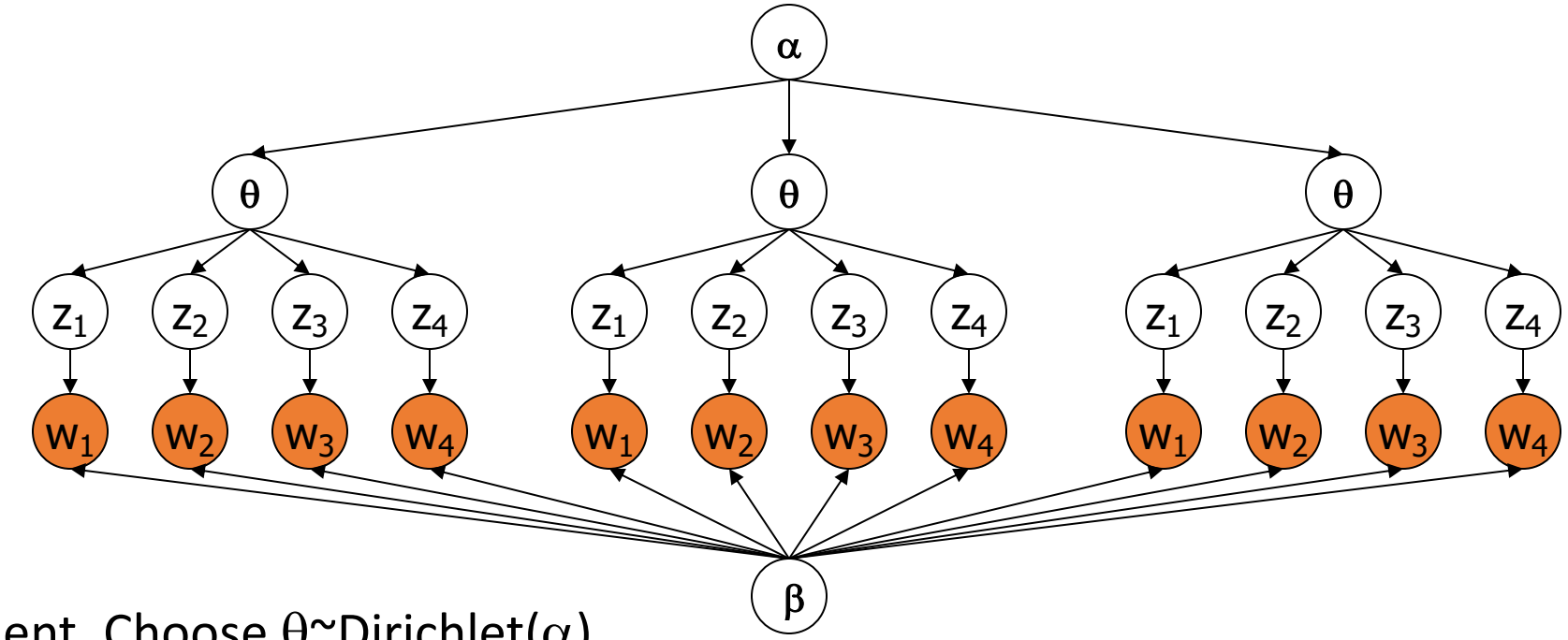
φ – word distribution for topic k

z – topic for the n-th word in document m

w – specific word



Step 6: LDA for Topic Modeling

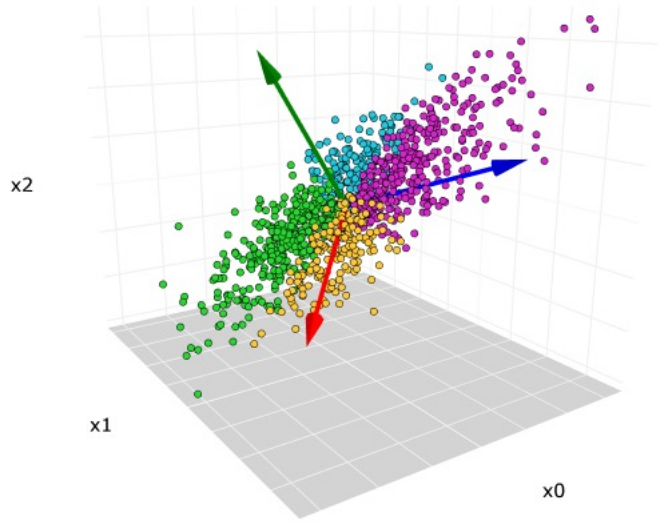


- For each document, Choose $\theta \sim Dirichlet(\alpha)$
- For each word:
  - Choose a topic z_n» Multinomial(θ)
  - Choose a word w_n from $p(w_n|z_n,\beta)$, a multinomial probability conditioned on the topic z_n.

The Similarity Between PCA and LDA

PCA is a dimensionality reduction technique, it is used for data having **numerical values**.

It is the linear combination of variables from which components are derived that are used to build the model. PCA works by breaking or decomposing a larger value (i.e. a singular value) into smaller values to reduce the dimensions.



The Similarity Between PCA and LDA

LDA operates in the same way as PCA does. LDA is applied to the **text data**. It works by decomposing the corpus document word matrix (the larger matrix) into two parts (smaller matrices): the Document Topic Matrix and the Topic Word. Therefore, LDA like PCA is a matrix factorization technique.